Aim: Advanced TypeScript using OOP and Data Persistence

PART 1:

Q1. What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm that model's software around "objects" rather than functions and logic. An object is a data field with unique attributes and behavior. OOP focuses on the objects developers want to manipulate, emphasizing principles like encapsulation, inheritance, and polymorphism to organize code more effectively, reduce complexity, and promote reusability.

Q2. What is a Class in programming?

A class is a fundamental building block in Object-Oriented Programming. It acts as a blueprint or prototype from which individual objects are created. A class defines:

- Attributes: The data members or variables that represent the state of an object.
- Methods: The functions or procedures that define the behaviors of an object.

Classes encapsulate data for the object and the methods to manipulate that data into a single, coherent unit.

Q3. What are Methods in a class?

Methods are functions defined within a class that describe the behaviors or actions that objects created from the class can perform. They operate on the object's internal data (attributes) to carry out specific tasks. Essentially, methods represent what an object can do.

Q4. What is the purpose of a Constructor?

A constructor is a special type of method in a class that is automatically called when a new instance of an object is created (instantiated). Its primary purposes are:

- To initialize the new object's attributes.
- To allocate any necessary resources for the object.
- To set up the object's initial state so it is ready for use.

Q5. What are Attributes in OOP?

Attributes, also known as properties, fields, or data members, are variables that belong to a class or an object. They represent the state, characteristics, or data of an object. Each object instantiated from a class has its own set of attributes, which hold values that differentiate one object from another.

Q6. Demonstrate the use of class along with methods, attributes, constructor and objects.

Code:

```
class Strawhats {
    public name: string | null;
    public role: string | null;
    public bounty: number | null;
```

```
    public age: number | null;

    constructor(name: string, role: string, bounty: number, age: number) {
        this.name = name;
        this.role = role;
        this.bounty = bounty;
        this.age = age;
    }

    showData(): void {
        console.log("\tName: " + this.name);
        console.log("\tRole: " + this.role);
        console.log("\tBounty: " + this.bounty);
        console.log("\tAge: " + this.age);
        console.log("--------------------------------------------------");
    }
}

const luffy = new Strawhats("Monkey D. Luffy", "Captain", 3000000000, 19);
const zoro = new Strawhats("Roronoa Zoro", "Swordsman", 1111000000, 21);
const nami = new Strawhats("Nami", "Navigator", 366000000, 20);
const usopp = new Strawhats("Usopp", "Sniper", 500000000, 19);
const sanji = new Strawhats("Sanji", "Cook", 1032000000, 21);
const chopper = new Strawhats("Tony Tony Chopper", "Doctor", 1000, 17);
const robin = new Strawhats("Nico Robin", "Archaeologist", 930000000, 30);
const franky = new Strawhats("Franky", "Shipwright", 394000000, 36);
const brook = new Strawhats("Brook", "Musician", 383000000, 90);
const jimbei = new Strawhats("Jinbe", "Helmsman", 1100000000, 46);

console.log()
console.log()
console.log("  ⣠⣴⣶⣄                    ⣠⣴⣶⣄  ")
console.log("⣠⣾⣿⣿⣿⣷⣄    ⣀⣠⣴⣾⣿⣿⣿⣿⣶⣦⣄⣀    ⣠⣾⣿⣿⣿⣷⣄")
console.log("⣿⣿⣿⣿
```

```
chopper.showData();
robin.showData();
franky.showData();
brook.showData();
jimbei.showData();
```

Output:

PART 2:

Q1. What Is Data Persistence?

Data persistence refers to the characteristic of data that outlives the execution of the program that created it. This means data is permanently stored and remains available even after the application is closed or the system is restarted.

Examples:

- ATM balance - Your bank balance remains the same between transactions
- Student records - Grades and enrollment information persist across semesters
- Login details - Usernames and passwords are stored for future use

Q2. List and explain Types of Data Storage

Method	Meaning	When to Use
Local Storage	Browser's temporary storage	Small web apps, temporary data
Indexed DB	Advanced browser database	Complex web applications, offline apps
Server Database	Centralized database server	Real-world applications, multi-user systems

Q3. What is an Entity?

An Entity is a TypeScript/JavaScript class that maps to a database table. Each entity instance represents a row in that table.

Q4. Create a program to demonstrate insertion and selection to/from a database safely using TypeScript and PostgreSQL

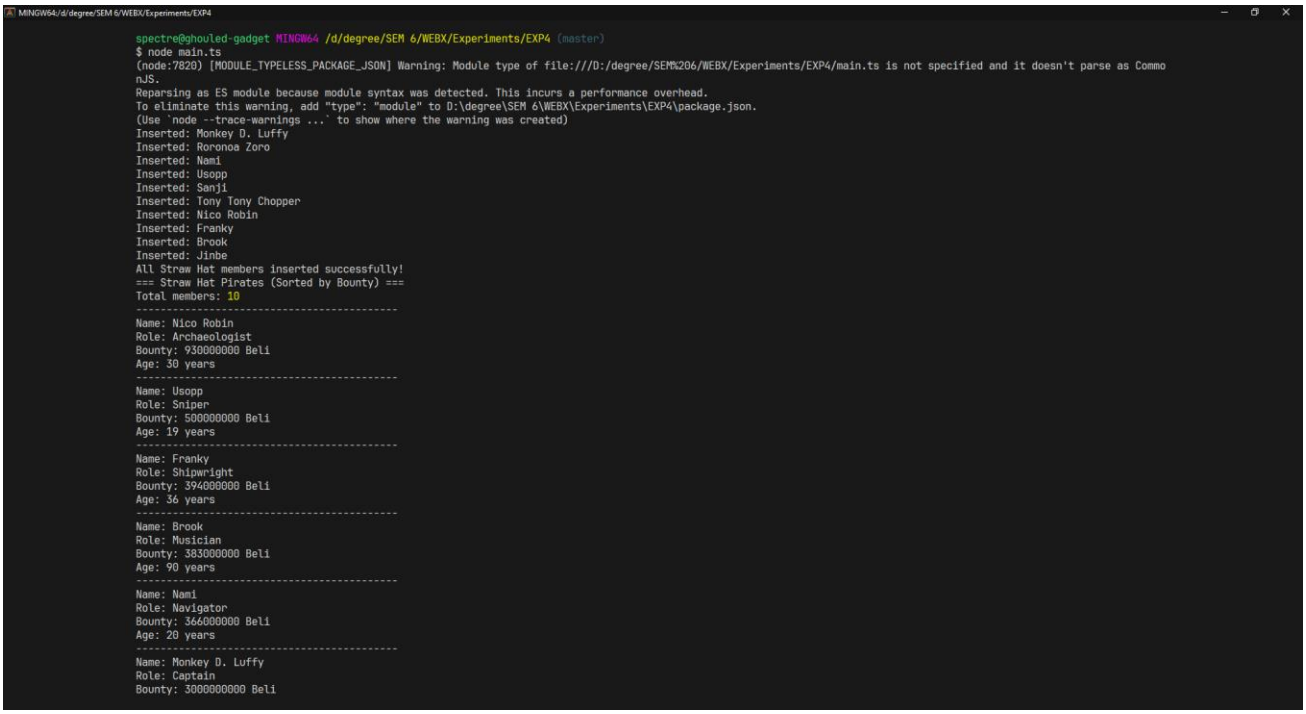
Code:

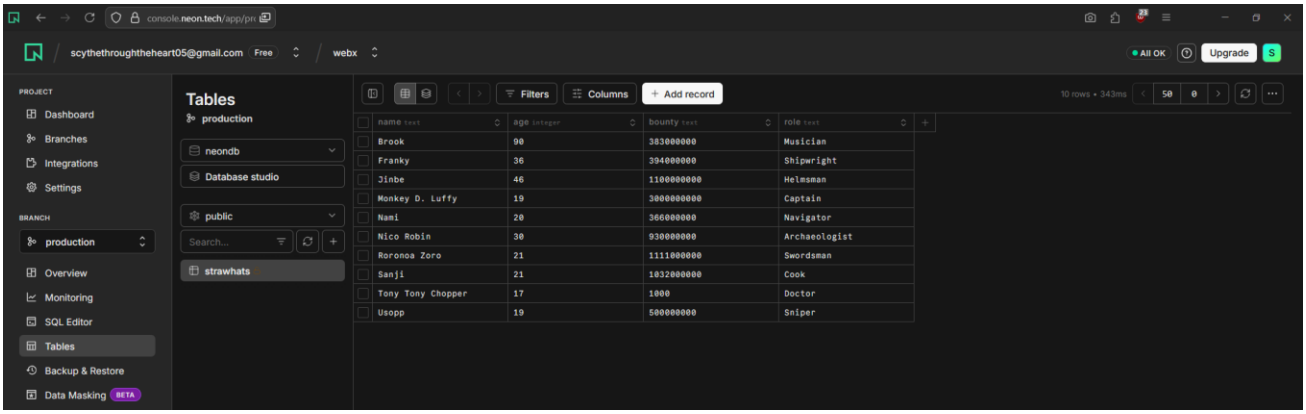
```
import { Pool } from "@neondatabase/serverless";

const connectionString = "postgresql://neondb_owner:npg_CJ3BPcaiey0E@ep-lingering-pine-
ais96quk-pooler.c-4.us-east-
1.aws.neon.tech/neondb?sslmode=require&channel_binding=require";
if (!connectionString) {
    throw new Error(
        "NEXT_PUBLIC_NEON_CONNECTION_STRING environment variable is not set",
    );
}

const pool: Pool = new Pool({
    connectionString: connectionString,
    ssl: true,
});
```

```
interface Strawhats {
    name: string | null;
    role: string | null;
    bounty: number | null;
    age: number | null;
}

async function insertStrawhats(): Promise<void> {
    const strawhats: Strawhats[] = [
        { name: "Monkey D. Luffy", role: "Captain", bounty: 3000000000, age: 19 },
        { name: "Roronoa Zoro", role: "Swordsman", bounty: 1111000000, age: 21 },
        { name: "Nami", role: "Navigator", bounty: 366000000, age: 20 },
        { name: "Usopp", role: "Sniper", bounty: 500000000, age: 19 },
        { name: "Sanji", role: "Cook", bounty: 1032000000, age: 21 },
        { name: "Tony Chopper", role: "Doctor", bounty: 1000, age: 17 },
        { name: "Nico Robin", role: "Archaeologist", bounty: 930000000, age: 30 },
        { name: "Franky", role: "Shipwright", bounty: 394000000, age: 36 },
        { name: "Brook", role: "Musician", bounty: 383000000, age: 90 },
        { name: "Jinbe", role: "Helmsman", bounty: 1100000000, age: 46 }
    ];

    try {
        for (const member of strawhats) {
            const query = `
                INSERT INTO strawhats (name, role, bounty, age)
                VALUES ($1, $2, $3, $4)
                ON CONFLICT (name) DO NOTHING;
            `;
            await pool.query(query, [
                member.name,
                member.role,
                member.bounty,
                member.age
            ]);
            console.log(`Inserted: ${member.name}`);
        }
        console.log("All Straw Hat members inserted successfully!");
    } catch (error) {
        console.error("Error inserting data:", error);
        throw error;
    }
}

async function getAllStrawhats(): Promise<Strawhats[]> {
    try {
        const result = await pool.query("SELECT name, role, bounty, age FROM strawhats
ORDER BY bounty DESC");

        console.log("=== Straw Hat Pirates (Sorted by Bounty) ===");
        console.log("Total members:", result.rows.length);
        console.log("------------------------------------------");

        for (const char of result.rows) {
            console.log(`Name: ${char.name}`);
            console.log(`Role: ${char.role}`);
            console.log(`Bounty: ${char.bounty} Beli`);
```

```
            console.log(`Age: ${char.age} years`);
            console.log("-----------------------------------------");
        }
        return result.rows;
    } catch (error) {
        console.error("Error fetching data:", error);
        throw error;
    }
}

await insertStrawhats()
await getAllStrawhats()
```

Ouput:





Conclusion:

In this experiment, we learned how TypeScript uses OOP to create secure and structured programs, and how data persistence helps store information permanently.